



Spring Framework Interview Questions and Answers

By Ramesh Fadatare (Java Guides)

(Key points from Udemy Course: [Java Developer Interview Mastery: The Only Course You Need](#))

What is Spring Framework?

Before Spring: The Problem

- Java applications were built in a **manual way**.
- Every class **created its own objects**, managed **configuration** and **lifecycle**.
- As applications grew:
 - Code became **tightly coupled** → changing one part required changing many others.
 - **Testing was difficult** → hard to replace objects with mocks.
 - **Repetitive and unorganized code** → especially in configuration and object wiring.
- Result: **Slower development, more bugs, high maintenance cost**.

What is the Spring Framework?

- A **Java framework** to build **clean, organized, and maintainable** applications.
- Provides a **structured system** to manage application components.
- Spring **standardizes**:
 - How objects are **created**,
 - How they **interact**,
 - How they are **configured**.
- In simple words: **Spring handles the complex setup**, and developers focus on **business logic**.

What Problem Does Spring Solve?

- Spring **reduces complexity** in Java applications.
- Automates:
 - **Object creation**
 - **Object lifecycle management**

- **Dependency management**
 - **Configuration**
- Encourages **loose coupling** by depending on **abstractions**, not concrete classes.
- Ensures even **large applications** stay **organized, predictable, and scalable**.
- Leads to **less code, fewer mistakes, and cleaner design**.

Why Spring Became Important

- Earlier enterprise Java technologies were:
 - **Heavy**
 - **Complicated**
 - **Difficult to manage**
- Spring introduced:
 - **Lightweight architecture**
 - **Layered and modular design**
 - **Faster development**
 - **Easier testing**
 - **Long-term maintainability**
- Because of this, Spring quickly became the **standard** for backend development in Java.

Benefits of Spring

- Helps write **clean and well-organized** code.
- **Reduces complexity** by handling object wiring and configuration.
- Spring applications are:
 - **Easy to test**
 - **Easy to extend**
 - **Easy to maintain**
- Provides a **consistent structure** across projects → team collaboration becomes easier.

- **Highly flexible:**
 - Supports **small apps**
 - **Large enterprise systems**
 - **Distributed microservices**
- Fits **real-world development needs** across industries.

Summary

- Spring is a **powerful and widely used** Java framework.
- Solves **tight coupling, complexity, and unstructured design**.
- Brings **stability, flexibility, and efficiency** to software development.
- Makes applications **clean, testable, scalable, and easy to maintain**.

Important Features of the Spring Framework

Dependency Injection (DI)

- Classes **do not create the objects** they depend on.
- **Spring creates the objects and provides them** where needed.
- Real-life example:
A driver does not build the engine. The manufacturer (Spring) provides it.
- Removes the need for the **new keyword** everywhere.
- Results in **cleaner, organized, and easily testable code**.

Inversion of Control (IoC)

- In normal Java: **Your code controls object creation**.
- In Spring: **Spring controls object creation and management**.
- Control is **inverted** from the application to the framework.
- Real-life example:
You order food, the kitchen prepares it. You don't control the cooking process.

Bean Management

- Objects created by Spring are called **Beans**.
- Beans are stored and managed inside the **Spring Container**.
- Think of the container as a **warehouse** that stores and supplies objects.
- Spring manages the **lifecycle** and **availability** of these Beans.

Annotation-Based Configuration

- Earlier Spring required **large XML configuration files**.
- Now we use annotations like:
@Component, @Service, @Repository, @Autowired
- These tell Spring **which classes to manage and where to inject dependencies**.
- Makes development **faster, cleaner, and simpler**.

Loose Coupling

- Spring encourages **programming to interfaces** instead of concrete classes.
- Example:
Depend on **NotificationService** interface, not Email/SMS implementation.
- If requirements change (e.g., switch SMS → WhatsApp),
only the implementation changes, not the entire application.
- Makes applications **flexible and scalable**.

Easy Unit Testing

- Since Spring **provides dependencies automatically**,
real objects can be **easily replaced with mock objects** during tests.
- Testing becomes **clean, easy, and isolated**.
- No need to modify the real code just for testing.

Centralized & Consistent Configuration

- Spring allows **centralized configuration** for the entire application.
- Database settings, service wiring, environment configs — **all organized**.
- Helps keep the project **clean and manageable**.

Lightweight & Modular

- Spring is **lightweight and modular**.
- You only use **the parts your application needs**.

- Example: If you only need Dependency Injection → Use **Spring Core only**.
- Results in **optimized performance**.

Declarative Support

- Use annotations to enable features like transactions, security, and caching
- Example: Add @Transactional to handle commit and rollback automatically
- Reduces repetitive logic and avoids errors

Summary

- Dependency Injection
- Inversion of Control
- Bean Management
- Annotation-Based Configuration
- Loose Coupling
- Easy Unit Testing
- Centralized Configuration
- Lightweight and Modular
- Declarative Support

Important Spring Framework Modules

- Spring is a **collection of different modules**; each module focuses on a **specific set of problems**.
- Spring is **modular** — you do not need to use everything; choose the modules your **project needs**.
- We will go **step-by-step** with **simple real-life examples**.

Spring Core Module

- **Base foundation** of the entire Spring Framework.
- Provides **Dependency Injection**.
- Spring **creates objects**, provides them to other objects, and manages their lifecycle.
- Before Spring: **new keyword**, **tightly coupled code**, **hard to maintain and test**.
- **Inversion of Control**: the **framework** takes control of **object creation**.

Spring Context Module

- **Extension of Spring Core**.
- Manages and organizes **Spring Beans**.
- Provides **ApplicationContext** (the **heart** of your Spring application).
- Solves **visibility and accessibility**: keeps track of **hundreds of beans** and **supplies the right bean** when needed.
- Like a **well-managed office storage room** (everything labelled and easy to find).

Spring AOP Module

- **Aspect-Oriented Programming**.
- Solves **repeating the same logic** in many places.
- Examples: **Logging every request**, **validating authentication**, **measuring performance**, **security checks**.
- Without AOP: **duplicate code**, **more mistakes**.

- With AOP: **write once**, Spring **applies it wherever needed**.
- Like **CCTV**: one system **watches everything**.

Spring JDBC Module

- **Java Database Connectivity** made simpler.
- Before Spring: **open connections, prepare statements, execute queries, handle exceptions, close everything** (30–40 lines for a small query).
- Spring JDBC provides **JDBC Template classes** — **repetitive code handled**.
- You **focus on SQL logic**; Spring **takes care of the rest**.

Spring ORM Module

- **Object Relational Mapping**.
- Integrates Spring with **Hibernate** and **JPA**.
- Solves **mismatch** between **database tables** and **Java objects**.
- Without ORM: **manual conversion** between SQL and objects (error-prone, time-consuming).
- With ORM: **interact with the database in terms of objects, easier and cleaner**.

Spring Transaction Management

- A **transaction**: multiple steps where **either all succeed or none do**.
- Example: **transfer money** — deduct from one account, add to another; if any step fails, **rollback**.
- Before Spring: **manual commit and rollback**.
- Use **@Transactional** — Spring **automatically handles commit and rollback**.
- **Reduces bugs** and ensures **data consistency**.

Spring MVC Module

- Used for **web applications and REST APIs**.
- Before Spring MVC: **Servlets and complex boilerplate**.
- Spring MVC uses **Controller classes, methods to handle HTTP requests, and automatic JSON conversion**.
- **Most commonly used module for building backend services**.

Spring Security Module

- **Security** is critical: **logins, user roles, permissions, session management, protection from attacks**.
- Implementing manually is **difficult and risky**.
- Spring Security provides **authentication and authorization out of the box and protects applications**.
- Ensures your application is **safe and controlled**.

Spring Boot (connects them all)

- **Not a replacement** for Spring; a **convenient layer on top of Spring**.
- **Removes complicated configuration** and helps **build applications quickly**.
- Think of Spring Boot as the **automatic version of Spring — less setup, fewer configuration files, much faster development**.

Summary

Spring Core & Context → **object creation and bean management**.

- **Spring AOP** → **cross-cutting logic** (logging, auth, performance, security checks).
- **Spring JDBC & ORM** → **simplify database access and object–table mapping**.
- **Spring Transaction** → **safe data operations with @Transactional**.
- **Spring MVC** → **web applications and REST APIs**.
- **Spring Security** → **authentication, authorization, protection**.

- **Spring Boot** → makes everything easier and faster.
- **Modular architecture** → **powerful, flexible, widely used** in real projects.

What Is Dependency Injection in Spring?

- Dependency Injection, often called DI, is a technique where objects do not create their own dependencies.
- Instead, Spring creates the dependencies and injects them into the classes that need them.
- This means your classes are not tightly connected to the objects they depend on.
- They simply declare what they need, and Spring provides it.
- Dependency Injection helps keep your code clean, modular, and very easy to test.
- It also allows you to change or replace implementations without modifying the core business logic.

What is Constructor-Based Dependency Injection in Spring?

- Constructor-based dependency injection is the technique where Spring injects dependencies through the class constructor.
- Instead of creating the dependency manually using the new keyword, Spring creates the dependency as a bean and passes it into the constructor automatically.
- This method is considered the recommended and safest form of dependency injection in Spring because it makes dependencies mandatory, explicit, and impossible to forget.
- In Spring, constructor injection supports `@Autowired`.
- If your class has only one constructor, Spring automatically injects the dependency even if you do NOT use `@Autowired`.
- If your class has multiple constructors, annotate the constructor you want Spring to use with `@Autowired` to make the selection explicit.
- `@Autowired` is optional for a single-constructor class and commonly used when multiple constructors exist.

What Is Setter-Based Dependency Injection?

- Setter-based dependency injection is a technique where Spring injects a dependency by calling a setter method on your class, instead of passing the dependency through the constructor.
- In simple words, Spring creates the object first, and then it calls the setter method to assign the dependency.
- The object is fully constructed, and after that, Spring injects what it needs.
- Setter injection is useful when a dependency is optional, or when you want the flexibility to replace that dependency later, such as during tests or custom configuration.
- Setter-based injection becomes active when you annotate the setter method with `@Autowired`.
- This annotation tells Spring, “whenever you create this bean, call this setter and inject the matching dependency.”

What Is the Spring IoC Container?

- The Spring IoC container is the core engine of the Spring Framework. It is responsible for creating objects, managing their lifecycle, injecting dependencies, and wiring everything together automatically.
- IoC stands for Inversion of Control. It means the control of object creation and dependency management is taken away from your application code and given to the Spring container.
- Instead of your classes creating objects manually using the `new` keyword, the IoC container creates those objects for you, stores them, manages them, and injects them wherever they are needed.
- These objects created and managed by the IoC container are called Spring Beans.
- When Spring starts your application, it scans your classes, finds classes annotated with `@Component`, `@Service`, `@Repository`, and `@Controller`, creates bean instances for them, and wires their dependencies automatically.

- This IoC container behavior makes Spring applications modular, testable, and loosely coupled.
- Spring provides two main IoC containers: the `ApplicationContext` and the `BeanFactory`.
- In Spring Boot applications, `ApplicationContext` is used by default because it provides advanced features like event publishing, internationalization, environment properties, and auto-configuration.

What Are the Types of Dependency Injection in Spring?

- In Spring, dependency injection is the mechanism that allows the framework to create objects and inject their required dependencies automatically.
- Spring supports three main types of dependency injections.

Constructor-Based Dependency Injection

- Constructor-based dependency injection is the preferred and recommended approach in modern Spring applications.
- Spring injects all required dependencies through the class constructor.
- This makes dependencies mandatory, ensures immutability, and guarantees that the object is fully initialized the moment it is created.
- Constructor injection works beautifully for unit testing because you can pass mock dependencies directly into the constructor.

Setter-Based Dependency Injection

- Setter-based dependency injection creates the object first and later injects the dependency by calling a setter method annotated with `@Autowired`.
- Setter injection is useful for optional dependencies or cases where you may want to change the dependency at runtime.

- The dependency is not mandatory for object creation, which gives flexibility but also means the object may exist before the dependency is assigned.

Field-Based Dependency Injection

- Field-based dependency injection injects dependencies directly into the field variable without using a constructor or setter method.
- You annotate the field with `@Autowired` and Spring assigns the dependency internally.
- This method is quick and convenient but not recommended because it makes unit testing harder and hides the class dependencies.
- Field injection also prevents immutability because fields cannot be final.

Which Is the Best Way of Injecting Beans and Why?

- In Spring, you can inject beans using constructor injection, setter injection, or field injection.
- **Constructor-based dependency injection is the best and most recommended way of injecting beans in Spring.**

Why Constructor Injection Is the Best

- Constructor injection makes dependencies mandatory.
- The moment your class is created, all required dependencies must be present.
- It prevents the object from being in a half-initialized state.
- It supports immutability because dependencies can be final and never changed later.

- It is the most test-friendly approach because you can pass mock objects directly into the constructor.
- Spring Boot automatically injects dependencies for single-constructor classes without `@Autowired`.
- Constructor injection detects circular dependencies immediately during startup.

Why Setter Injection and Field Injection Are Not the Best

- Setter injection is used only when the dependency is optional or configurable.
- It is flexible but allows objects to exist without required dependencies, which makes it unsafe for mandatory dependencies.
- Field injection is the weakest approach.
- It hides dependencies inside the class, breaks immutability, and makes unit testing difficult.
- That is why field injection is strongly discouraged in modern Spring development.

What Is the Difference Between BeanFactory and ApplicationContext in Spring?

- Both BeanFactory and ApplicationContext are IoC containers in Spring.
- Both manage beans, create objects, wire dependencies, and control the bean lifecycle.
- But ApplicationContext is the **more advanced, feature-rich, and commonly used** container.

- BeanFactory is the **basic, lightweight** container used only in rare, low-level scenarios.

BeanFactory (Simple, Basic Container)

- BeanFactory is the simplest IoC container in Spring.
- It loads beans lazily — beans are created only when `getBean` is called.
- It performs only dependency injection and bean creation.
- It does **not** support internationalization.
- It does **not** support event publishing.
- It does **not** support AOP auto-proxying by default.
- It does **not** support Spring Boot features.

ApplicationContext (Full-Featured Container)

- ApplicationContext is the full-featured IoC container.
- It eagerly loads beans at application startup, making runtime faster.
- It supports internationalization and event handling.
- It supports annotation scanning and Spring Expression Language.
- It supports environment properties and lifecycle callbacks.
- It supports AOP auto-proxying and advanced Spring features.
- Spring Boot uses ApplicationContext internally for auto-configuration, bean scanning, Actuator, and the full Spring runtime environment.

What Spring Sub-Projects Do You Know? Describe Them Briefly

Spring Core

- Spring Core is the foundation of the entire Spring ecosystem.
- It provides the IoC container and dependency injection features.
- All other Spring modules are built on top of Spring Core.

Spring MVC

- Spring MVC is used to build web applications and REST APIs.
- It follows the Model-View-Controller architecture.
- It provides controllers, request mapping, data binding, interceptors, and validation.

Spring Boot

- Spring Boot is the modern entry point into the Spring ecosystem.
- It reduces manual configuration through auto-configuration and starter dependencies.
- It includes an embedded server and helps build production-ready applications quickly.

Spring Data

- Spring Data simplifies database access.
- It provides repository interfaces that remove boilerplate CRUD code.
- It supports JPA, MongoDB, Redis, Cassandra, Elasticsearch, and more.

Spring Security

- Spring Security provides authentication and authorization.
- It supports password management, JWT, OAuth2, and protection against common attacks.
- It integrates closely with Spring Boot and Spring MVC.

Spring Cloud

- Spring Cloud provides tools for cloud-native and microservice architectures.
- It offers service discovery, API gateway, load balancing, tracing, fault tolerance, and centralized configuration.
- It makes microservices production-ready.

Spring AOP

- Spring AOP provides aspect-oriented programming.
- It separates cross-cutting concerns like logging, security, and transactions.
- It works with the IoC container to keep code clean and maintainable.

Spring Batch

- Spring Batch is used for large-scale batch processing.
- It supports chunk processing, retries, transaction management, and job monitoring.
- It is widely used in enterprise and financial systems.

Spring Integration

- Spring Integration supports application-to-application messaging.
- It implements enterprise integration patterns with channels, routers, and transformers.
- It integrates with JMS, Kafka, RabbitMQ, and other messaging systems.

Spring WebFlux

- Spring WebFlux provides reactive, non-blocking programming.
- It is built on Reactor and supports massive concurrency and high throughput.
- It is designed for scalable and event-driven applications.

What Is the Default Bean Scope in the Spring Framework?

- In Spring, every bean is managed by the IoC container, and each bean lives inside a specific scope.
- When you don't define a scope, Spring applies the **default scope**.
- The **default bean scope in the Spring Framework is the singleton scope**.
- This means Spring creates only **one instance of the bean** for the entire application context.
- Every time another class needs this bean, Spring returns **the same shared instance**.

How Singleton Scope Works

- When the application starts, Spring scans for beans and creates one instance of each singleton bean.
- Spring stores this instance inside the **ApplicationContext**.
- All controllers, services, and components that depend on this bean get **exactly the same object**.
- This improves performance, reduces memory usage, and ensures consistent shared behavior across the entire application.